Startup Studio · Columbia University · Fall 2026

Usability testing and AI for your work

The rest of the semester, more product testing, and AI system breakdown

Class 8 · Friday, October 30, 2026 · 1:10 to 3:40 PM · Pupin 420

What we're doing today

1:10 PM

The rest of the semester

Presentations, the leaderboard, the weekly update.

1:21 PM

Claude Code

How it works, the four layers, things that bite.

2:01 PM

Two more ways to test

Pirate metrics, expert review, usability testing.

2:26 PM

Studio Time

Write a script, then two rounds of usability testing.

3:16 PM

Homework

What's due next Friday.

The rest of the semester, 1 of 2

Date	Class	Topic	Homework due
Today	Class 8	What to build next	HW 7: MVP learnings
Nov 6	Class 9	Get users and analytics	HW 8: roadmap and CLAUDE.md
Nov 13	Class 10	Cold start	HW 9: growth plan, analytics
Nov 20	Class 11	Showcase and fundraising	HW 10: cold start strategy

The rest of the semester, 2 of 2

Date	Class	Topic	Homework due
Nov 27	No class	Thanksgiving	
Dec 4	Class 12	Feature freeze and pitching	HW 11: iterate, then freeze
Dec 11	Class 13	Unit economics, deck workshop	HW 12: pitch deck draft 1
Dec 18	Demo Day	Final pitches and live demos	HW 13: final deck

Team presentations, every class

Format

- 10 minutes to present: the live product and your data
- 5 minutes for questions and feedback
- 1 or 2 teams every class, starting Nov 6

Why

- Practice reps before Demo Day
- Real feedback from the whole room

Sign up today. Earlier means more time to use the feedback. Later means more data to show.

User count leaderboard

We track this every class. A real user signed up and did the core action once.

Team	Real users

Why the number matters

Your grade

10%

User growth is a tenth of the course grade

Per point

10

Real users earn 1 point

Full credit

100

Real users or more earns every point

Weekly update

Every team submits a short update each week. It's due before class on Friday.

- **Accountability:** What we built last week and why
- **Planning:** What we're building this week and why
- **Reporting:** Current user count
- **Issues:** Biggest blocker (if any)

Half a page. Not a novel.

Weekly update: the file


```
teams/your-team-name/weekly_update_YYYY-MM-DD.md
```

Two kinds of AI workflows

AI for your work

- Tools that help you build faster
- Claude Code, skills, agents, CLAUDE.md

AI in your product

- AI features you ship to users
- Summarization, search, recommendations, agents

Today is the first one.

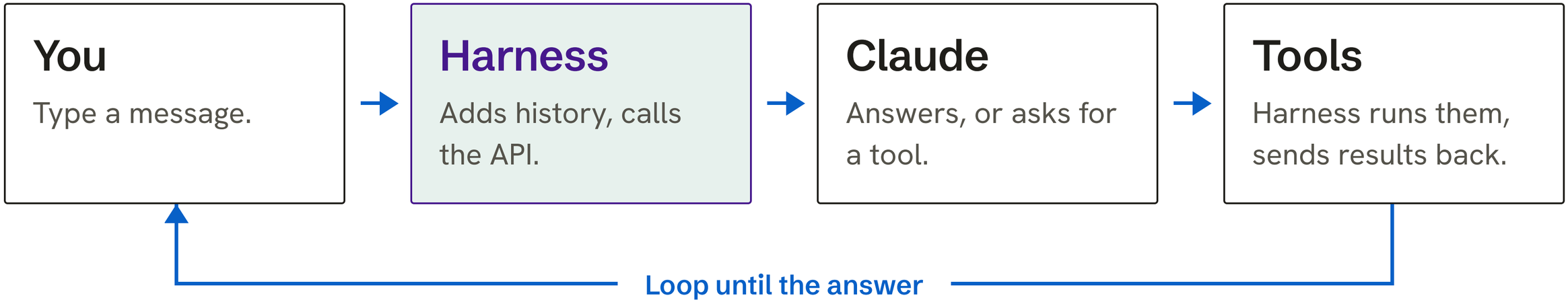
Part 2

How Claude Code actually works

You've used it for weeks. Let's open the hood.

The agentic loop

Claude Code is a conversation loop that wraps the Claude API.



The harness sits in the middle of every step.

What happens in the loop

01	**Fresh every turn**	Every turn is a stateless API call. The harness replays the full history each time.
02	**Tools are schemas**	Claude outputs a request like read_file for app.py. The harness runs it and sends back the result.
03	**The loop continues**	It runs until Claude has nothing left to do. A complex task might take 10 to 20 loops or more.

The harness is the product

What you're really using

What people call Claude Code being great at coding is the harness: file access, tool permissions, context management. The model is the same one in claude.ai. The wrapper is the product.

The model vs. the harness

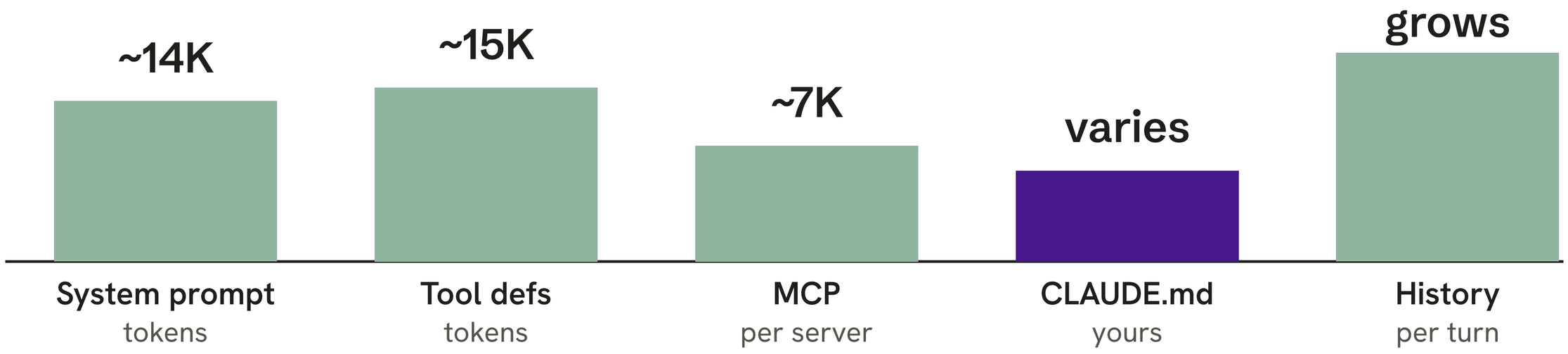
The model (LLM)

- **Stateless.** No memory between turns
- Can't read files, run code, or browse
- Just predicts the next token
- Identical in claude.ai, Claude Code, Cursor, Copilot

The harness (Claude Code)

- Stores and **replays** conversation history
- Executes tools (Read, Edit, Bash, MCP)
- Manages permissions and safety
- **This is the product.** This is what differs.

What's in every API call



Every turn

36K+

tokens already spoken for before you type a word

The full payload is re-sent with every message. This is why prompt caching exists.

Claude Code configuration layers research, March 2026

MCP bloat is real

Tools cost tokens

Developers have reported 66,000 to 144,000 tokens used by MCP tools alone. Claude Code answers with deferred tools: it loads only 3 to 5 relevant schemas per query, about 85% less.

The context window illusion

What you think

200K

Tokens of space for your work

What you get

150K

Usable tokens before compaction

Why a lean setup matters

- **Compaction** triggers at ~95% full. It is lossy.
- More overhead means less room before compaction.
- A bloated CLAUDE.md doesn't just cost money. It **steals working memory**.
- **Context rot:** accuracy drops as context grows, even before the limit.
- **Lost in the middle:** models attend most to the start and end.

The configuration layers

Part 2

Tuning the engine.

Four mechanisms

Mechanism	Loaded	Compliance	Use for
CLAUDE.md: always-on context	Every API call	High (primacy)	Identity, rules
Skills: on-demand workflows	When invoked	Medium (middle)	Workflows, reference
Agents: isolated workers	When spawned	Highest (fresh window)	Parallel tasks
Hooks: shell scripts	Every trigger	100% (deterministic)	Non-negotiable rules

Why skills get followed less

Lost in the middle

Skills load mid-conversation. LLMs pay less attention to the middle of context. That's why skill instructions get followed less reliably than CLAUDE.md.

What goes where

01	**Every conversation?**	Does it apply to every conversation? Yes: CLAUDE.md. No: keep going.
02	**Multi-step workflow?**	Is it a workflow with human gates? Yes: a skill. No: keep going.
03	**Runs on its own?**	Can it run autonomously, in isolation? Yes: an agent. No: probably a skill.
04	**No exceptions?**	Must it run every time, no exceptions? Yes: a hook (deterministic).

What the pros put in CLAUDE.md

Author	Lines	What's in it
Simon Willison	30	Build commands, minimal instructions
Anthropic (their own repo)	51	Commands, architecture, things that bite
Boris Cherny	59	Behavioral rules distilled into principles
Dan Abramov (React)	74	Architecture, dev commands, conventions
Andrej Karpathy	72	Think before coding, simplicity, surgical changes

Ten published CLAUDE.md files, collected March 2026

Things that will bite you

Part 2

Three things LLMs can't do, and what fixes each one.

LLMs cannot do math

They don't calculate. They predict the next likely token.

What happens

- Ask: what's 17% of 4,823?
- You get a confident wrong number
- It looks right. It's not.

The fix

- calc-mcp, an MCP server
- "use python for any math"
- Every calculation routes to its math tool
- Math, dates, unit conversions

The fix lives in CLAUDE.md

An always-on rule plus an MCP server.

```
# From a CLAUDE.md
All math and date calculations must use the calc-mcp tool.
Do not do mental math. Do not estimate.
```

LLMs don't know what day it is

Dates and date math are often bad

What happens

- Ask: schedule this for next Tuesday
- The LLM doesn't know today's date
- It will guess or make one up

The fix

- A hook runs a Python script on every message
- The output is added before Claude sees your prompt

The fix is a hook

Deterministic, not probabilistic. It fires every time.

```
# settings.json
"UserPromptSubmit": [{
  "hooks": [{
    "type": "command",
    "command": "python3 dateutil.py --now"
  }]
}]
-> Friday October 30, 2026 - 1:10 PM
```

LLMs can't touch the real world

Without tools, an LLM is a brain in a jar. Tools and MCP fix that.



Browser testing

A Playwright skill writes and runs browser automation. Tests pages, fills forms, takes screenshots.



Browser control

Chrome MCP. Navigate real tabs. Read pages, click, fill forms. Live interaction.



External services

MCP servers for Gmail, Google Calendar, and Todoist. Read and act on your behalf.

The second half

You've validated problems and built MVPs.

In startup terms, your product is now going into beta.

Your goals for the rest of the semester

You have a working product. Now make it better, and get people using it.

Make the product better

- Add features
- Refine the ones you have

Acquire users

- Get real people using it
- The leaderboard tracks it every class

How do you decide where to focus? A framework: pirate metrics.

Pirate metrics

Part 1

AARRR (matey)

Without a framework, founders diagnose by guessing:

01 **More marketing?**

You say "We need more marketing." Maybe. Or maybe 90% of people who find you leave without getting the point.

02 **Not sticky?**

You say "The product isn't sticky." Maybe. Or maybe users who reach the aha moment DO come back, but almost no one is reaching it yet.

03 **No demand?**

You say "Nobody wants this." Maybe. Or maybe the landing page is just terrible and the product is fine.

AARRR is a diagnostic map.

It helps you figure out where to focus your effort.

AARRR: the five stages

Stage	The question	What you do
Acquisition	Are people finding you?	Drive traffic, any channel
Activation	Are they reaching the aha moment?	Optimize onboarding
Retention	Are they coming back?	Deliver ongoing value
Referral	Are they telling others?	Build in sharing / delight
Revenue	Are they paying?	Monetize the value delivered

AARRR: what you measure

Stage	What you measure
Acquisition	Visitors, signups, traffic source
Activation	% completing key first action
Retention	DAU/WAU, return rate, churn
Referral	Referral rate, NPS, viral coefficient
Revenue	MRR, LTV, conversion rate

Each stage is a gate. You can't skip one and hope for the best.

Activation is the gate

Scenario A: 5% activation

- 1,000 visitors a month
- × 5% activation rate
- = 50 active users

Scenario B: fix activation first

- 1,000 visitors a month
- × 50% activation rate
- = 500 active users

Same traffic. 10× the active users. The lever is activation, not acquisition.

Activation

Fix onboarding before scaling traffic.

Otherwise you're filling a leaky bucket. You can't retain someone who never got the point.

Your aha moment

When does a user first understand why your product exists?

Slack	"I can message my whole team in one place and search it later"
Airbnb	"Real people's apartments: cheaper and more interesting than a hotel"
Dropbox	"My file is just... there, on my other computer"

Other frameworks: North Star Metric

- One number that captures your core value.
- Airbnb: nights booked. Spotify: time spent listening.
- Forces focus, but doesn't tell you where you're leaking.

Other frameworks: HEART (Google)

- Happiness, Engagement, Adoption, Retention, Task Success.
- More user-experience-focused than growth-focused.
- Better for mature products optimizing quality, not early-stage product-market fit.

Other frameworks: Jobs-to-be-Done

- Focuses on why users hire your product, not what they do in it.
- Great for discovery and positioning.
- Doesn't give you the funnel metrics to diagnose where users drop off.

So how do we know if it's working?

You know what activation is. How do you find out if yours works?

User testing

- Watch real people try to use your product.
- See where they get confused, and whether they reach the aha moment.
- Answers: WHY they get stuck

Analytics

- Measure what happens at scale with GA4 and Amplitude.
- See how many people drop off at each step.
- Answers: HOW MANY get stuck

These two methods feed into each other. We'll cover both today.

User testing

Part 3

Two ways to test

Expert review

- Someone who knows your industry or knows UX well walks through your product.
- Fast. Finds the obvious problems.

Usability testing

- A real user tries a task and thinks out loud. You observe silently.
- Can they use it as you intended? What do they do that you didn't intend?

Today we use usability testing to speed up activation: getting people to your aha moment.

You don't need 50 testers

85%

of usability problems show up with just 3 to 5 testers

More users after 5 gives diminishing returns. The most obvious problems get hit over and over.

Nielsen Norman Group, Why You Only Need to Test with 5 Users

You don't need 50 testers

- For UX, run a small number of tests and iterate rapidly.
- Don't aim for comprehensive coverage or statistical significance.

Find 5 people this week. Start now.

Strangers > friends

Friends will be nice

- Fill in your gaps with imagination
- Already know your product from you
- Won't tell you when something is confusing
- Their social desirability bias is your blind spot

Strangers will struggle

- Arrive completely cold, like real users
- Won't apologize for confusion
- Click the wrong things without explaining
- Their confusion IS the data

How to find people

Reddit

r/[relevant community] or r/beermoney. "20-min test, $10 Venmo"

Craigslist gigs

Same offer. Title: "Paid usability test, 20 min"

Discord or Slack

Communities where your actual target users hang out

How to find people

Cold DMs

LinkedIn, Instagram, Twitter: wherever your target users are

Columbia network

Students and staff you don't know personally

Friends or classmates

OK to start. Lower quality data. Fine for session 1.

User testing

Offer $10 per session.

Venmo, PayPal, or a gift card. Paying dramatically increases responses from strangers. Optional.

Consent form

- What the session is (user testing to improve a product)
- How notes will be used (to improve the product, shared with the class)
- No personal data stored or shared
- Right to stop at any time
- Recording consent (if applicable)

How to run the session

01	**Brief them**	"Please think out loud. I won't help you. There are no wrong answers."
02	**Watch and listen**	Hand over the laptop or URL. Give them your tasks, or let them explore.
03	**Don't help**	Let them struggle. If they're stuck: "What are you looking for?" Then stop.
04	**Ask after**	When did you get it? What was confusing? Would you use it again? One magic-wand change?

Note behavior, not opinion

✕ Not this (opinion)

- "She found it confusing."
- "They liked the concept."

✓ This (behavior)

- "She clicked the X three times."
- "He paused 8 seconds on sign-up."

Look for hesitations, wrong clicks, questions out loud, dropped tasks, workarounds. Record the screen.

Interview script template: intro

Start from this. Customize the tasks for your product.

Read this verbatim

"I'm going to have you try to use our product. Please think out loud: say whatever's on your mind. I won't help you. There are no wrong answers. We're testing the product, not you."

Interview script template: tasks

- 1. "Go to [URL]. What do you think this product does?"
- 2. "Please try to sign up."
- 3. [A task that leads to your aha moment] "Create your first [thing]"

Task 3 is yours to write. It should lead directly to your aha moment.

Interview script template: after

1. "When did you first understand what this is for?"
2. "What was the most confusing part?"
3. "What would have made it easier?"
4. "Would you use this again? Why or why not?"

How this works

2 people: host

- Stay at your station
- One person reads the script, one takes notes
- Don't help. Note behavior, not opinion.

2 people: test

- Go to your partner team's station
- Do the tasks they give you
- Think out loud the whole time

Round 1 pairings

Similar products paired on purpose. You learn most from teams like yours.

Pair	Team	Team	What they share
1	[Ken: team]	[Ken: team]	[Ken: theme]
2	[Ken: team]	[Ken: team]	[Ken: theme]
3	[Ken: team]	[Ken: team]	[Ken: theme]
4	[Ken: team]	[Ken: team]	[Ken: theme]

Write your script, then test

Round 1, 25 minutes

25:00

1. **First:** 2 or 3 tasks from the template, ending at your aha moment.
2. **Hosts:** brief the tester, read the tasks, then step back. Don't help.
3. **Testers:** do the tasks and think out loud.
4. **Note-taker:** pauses, wrong clicks, questions out loud, dropped tasks.
5. After: ask your post-session questions.

Switch roles and partners

Round 2, 20 minutes

20:00

1. Hosts become testers. Testers become hosts.
2. Pair with a team from a different category.
3. **Hosts:** brief, read the tasks, step back. Don't help.
4. **Testers:** do the tasks and think out loud.

Quick debrief

- What **surprised** you?
- Where did people get **stuck**?
- Did anyone get to your **aha moment**?

Next week: how to decide what to fix first.

Homework

Usability testing with real people, and your first weekly update.

What to submit by Friday, November 6

- **Usability testing:** 3 to 5 sessions with real people. Strangers over friends.
- Use your script from today. Record with consent.
- **Write up:** what you observed, what you're changing, and why
- **Weekly update:** your first one, in your team folder

Questions?